

1. Introduction

這次作業要實測 Deep Q-Network 在不同環境中的表現。在 Task 1 使用 CartPole-v1 作為低維度狀態空間的測試環境，實作基本的 fully connected DQN。Task 2 將 DQN 使用到 Atari Pong-v5，使用影像前處理與 CNN 處理視覺輸入。Task 3 則在 Pong-v5 上加入 Double DQN、Prioritized Experience Replay 與 multi-step return，以改善 vanilla DQN 的 sample efficiency。

本作業的主要發現是：vanilla DQN 可以在相較簡單的 CartPole 環境快速收斂，但在 Pong 這類高維度環境中需要大量 environment steps 才能學到穩定策略。加入 Double DQN、PER 與 multi-step return 後，模型能更有效利用 replay buffer 中的重要 transition，並且更快傳遞 reward information，因此在相同 environment steps 理論上應通常能達到更高的 evaluation score。

2. Implementation

甲、Vanilla DQN on CartPole

由於 CartPole 的 state 是四維輸入的所以用 fully connected network 來近似 Q-function。我的 DQN 架構包含兩層 hidden layers，每層有 128 個 neurons，activation function 使用 ReLU，最後輸出每個 action 對應的 Q-value。訓練時使用 epsilon-greedy policy 進行 exploration。每次更新時，從 replay buffer 中 uniform random sampling 一個 mini-batch，計算 Bellman target 與目前 Q-value 之間的 loss，最後使用 Adam optimizer 更新 online Q-network。

在 DQN 中，Bellman error 用來衡量目前 Q-network 的預測值與 Bellman target 之間的差距。對於 replay buffer 中取出的 transition (s, a, r, s', done)，我先使用 online Q-network 計算目前 state s 下執行 action a 的 Q-value。接著使用 target network 計算下一個 state 的最大 Q-value，並建立 Bellman target，若該 transition 已經到達 terminal state，則不再加入下一狀態的 Q-value，因此 target 只會等於 immediate reward。最後 Bellman error 為 target 與目前 Q-value 的差距，loss 則用來最小化這個誤差。在 Task 1 中我使用 MSE loss，而在 Pong 相關任務中則使用 SmoothL1loss，以提高訓練穩定性。

乙、Vanilla DQN on Pong

Task 2 中，我將 DQN 從 CartPole 擴展到 Atari Pong-v5。與 CartPole

不同，Pong 的 observation 是 RGB 影像，因此不能直接使用 fully connected network 處理原始輸入。我先對影像進行預處理，包括將 RGB 影像轉成 grayscale、resize 成 84x84，並堆疊連續 4 張 frame 作為模型輸入。

使用 frame stacking 的原因是，單張影像只能提供物體的位置資訊，但無法判斷球與球拍的移動方向。透過堆疊連續 frames，讓 agent 可以從影像變化中推測速度與移動方向。

在模型架構上，我使用 CNN-based DQN。CNN 前半段負責從 4 張 84x84 frame 中抽取視覺特徵，後半段使用 fully connected layer 輸出每個 action 的 Q-value。訓練流程仍維持 vanilla DQN 的架構，包括 epsilon-greedy exploration、uniform replay buffer、target network，以及定期更新 target network。

丙、Enhanced DQN on Pong

i. Double DQN

在 Task 3 中，我將 vanilla DQN 更改成 Double DQN。Vanilla DQN 在計算 target 時，會直接使用 target network 對下一個 state 的所有 action Q-value 取最大值。然而，max operator 容易造成 Q-value overestimation，因為被選到的 action 可能只是因為估計誤差而偏高。

我更改成 Double DQN 的想法是將 action selection 與 action evaluation 分開。先使用 online Q-network 選出下一個 state 中 Q-value 最大的 action，再使用 target network 評估這個 action 的 Q-value

在我的實作中，online Q-network 負責產生 next_actions，而 target network 再根據 selected actions 取出對應的 next_q_values。這樣可以降低 vanilla DQN 中 Q-value overestimation 的問題，使訓練更穩定。

ii. PER memory buffer

在 Task 3 中，我將原本的 uniform replay buffer 改為 Prioritized Experience Replay。傳統 replay buffer 會從所有 transitions 中等機率隨機抽樣，但這樣可能會浪費許多更新在已經學得很好的樣本上。PER 則可以根據每筆 transition 的 Bellman error 設定 priority，讓 TD error 較大的 transition 有較高機率被抽到。

在程式中，Prioritized Replay Buffer 包含三個主要結構：buffer 用來儲存 transition，priorities 用來儲存每筆 transition 的 priority，pos 則記錄目前 circular buffer 的寫入位置。當新的 transition 被加入時，如果尚未計算 TD error，則先給它目前 buffer 中最大的 priority，確保新的資料有機會被抽到。最後，每筆樣本的 loss 會乘上對應的 weight，再取平均作為最終 loss。完成一次 gradient update 後，我會根據新的 TD error 更新該 batch 中樣本的 priority。

iii. Multi-step return

在 vanilla DQN 中，target 只使用 1-step return，也就是 immediate reward 加上下一個 state 的 bootstrap value。這種方式在 reward sparse 或 delayed reward 的環境中可能學得較慢，因為 reward information 需要很多次 update 才能往前傳遞。因此在 Task 3 中，我加入 multi-step return。我設定 n-step = 3，並暫存最近的 transitions。當 buffer 中累積到 3 筆 transition 後，我會計算折扣後的 reward，接著將這個 n-step transition 存入 replay buffer。這樣可以讓 reward information 更快地往前傳遞，這樣或許對 Pong 這類 reward 較 sparse 的環境尤其有幫助。

3. Experimental Results and Discussion

甲、Training curves



圖 1 顯示 Task 1 在 CartPole-v1 環境中的 evaluation reward 曲線，其中 x 軸為 Env Step Count，y 軸為 Eval Reward。從圖中可以觀察到，訓練初期 agent 的 reward 較低且表現不穩定，這是因為模型一開始尚

未學到有效 policy，同時 epsilon-greedy 策略仍保留較高比例的隨機探索。隨著 environment steps 增加，evaluation reward 整體呈現上升趨勢，表示 DQN 逐漸學會如何控制 cart 以維持平衡。雖然中間仍出現數次波動，但在訓練後期 evaluation reward 最終達到 500，代表 agent 已能穩定完成整個 episode。這說明 vanilla DQN 對於 CartPole 這種低維度、reward signal 明確的控制任務具有良好的學習效果。

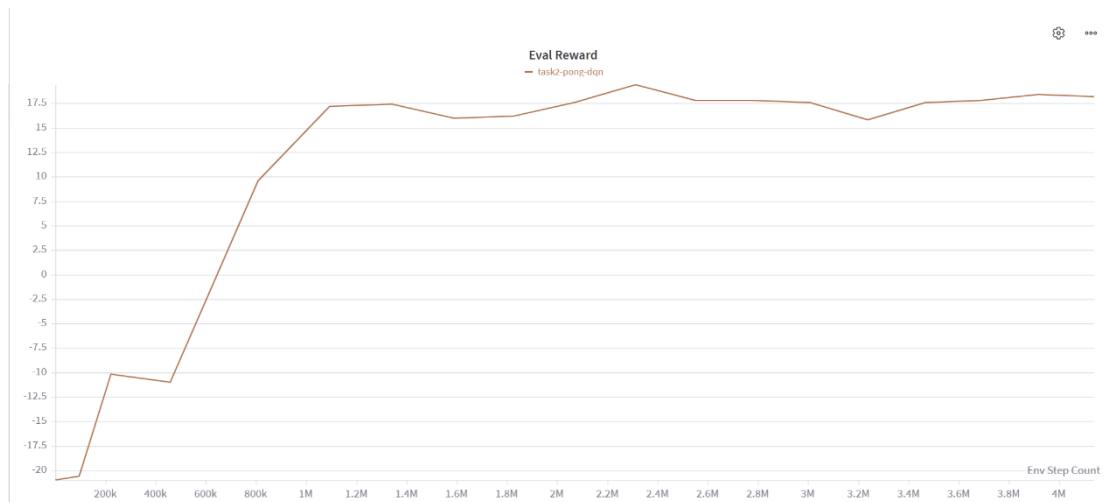


圖 2 顯示 Task 2 在 Pong-v5 環境中使用 vanilla DQN 的 evaluation reward 曲線，其中 x 軸為 Env Step Count，y 軸為 Eval Reward。訓練初期 agent 的 evaluation reward 約為-20，代表模型幾乎無法有效回擊球，表現接近隨機策略。隨著訓練進行，reward 在約 0.5M 到 1.1M environment steps 之間快速上升，表示 CNN-based DQN 開始從影像輸入中學到球、球拍與運動方向之間的關係。

在 1M environment steps 之後，evaluation reward 大多維持在 16 到 19 左右，代表 agent 已能穩定贏過對手，並且具有良好的策略。雖然曲線仍有小幅波動，但整體表現相當穩定。這說明即使 Pong-v5 是高維度影像輸入任務，透過 CNN-based DQN，vanilla DQN 仍能在足夠的 environment interactions 後學到有效策略。

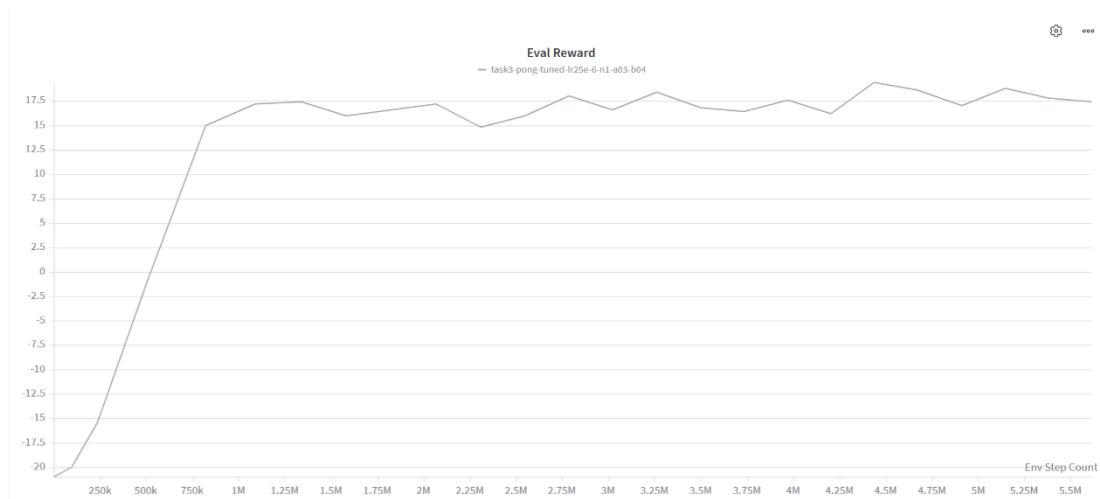


圖 3 顯示 Task 3 在 Pong-v5 環境中使用 enhanced DQN 的訓練曲線。此版本在 vanilla DQN 的基礎上加入 Double DQN、Prioritized Experience Replay (PER) 與 multi-step return，希望能改善原先訓練不穩定的問題並重新調整 hyperparameters，使訓練過程更加穩定。從圖中可以觀察到，Task 3 在訓練初期 evaluation reward 接近 -20，表示 agent 一開始仍接近隨機策略，幾乎無法有效回擊球。隨著 environment steps 增加，reward 在約 0.5M 到 0.9M environment steps 之間快速上升，並在約 1M steps 後達到 15 分以上。這表示 enhanced DQN 已經能從 frame-stacked image observations 中學到有效的策略。

在中後期，Task 3 的 evaluation reward 大多維持在 16 到 19 之間，曲線相較原本設定更加穩定，沒有出現長時間掉回低分區間的情況。這顯示經過 hyperparameter tuning 後，Double DQN、PER 與 multi-step return 的組合能夠有效提升 Task 3 的學習穩定性。雖然 reward 仍有小幅震盪，但整體表現已接近 Task 2 的 vanilla DQN，並且明顯優於原先未調整超參數的 Task 3 結果。

乙、Sample efficiency comparison

從 sample efficiency 的角度來看，Task 1 在 CartPole-v1 上最容易學習，因為環境狀態為低維度數值資料，因此 agent 能在相對較少的 environment steps 中學到有效策略，最後達到 500 分。

在 Pong-v5 中，Task 2 的 vanilla DQN 表現出較佳的 sample efficiency。模型在約 0.8M 到 1.1M environment steps 之間快速從負分提升到 10 分以上，之後穩定維持在 16 到 19 分左右。這表示 Task 2 的 CNN-based DQN 能有效利用收集到的樣本，逐漸學到穩定的策

略。

相較之下，Task 3 雖然加入 enhanced techniques，但 reward 上升速度較慢，且最終表現低於 Task 2。Task 3 約在 1M environment steps 後才進入正分區間，後續雖然偶爾達到 10 分以上，但整體仍低於 Task 2。根據本次實驗結果，Task 2 的 vanilla DQN 在 Pong-v5 上具有較好的 sample efficiency，而 Task 3 的 enhanced DQN 可能需要進一步調整超參數才有可能發揮想像中的優勢。

丙、Ablation study

Experiment	Double DQN	PER	n-step return	Hyperparameter tuning	Mean Reward	Observation
Task 2	No	No	No	Yes	19.30	通過 requirement，表現穩定
Task 3 600k	Yes	Yes	Yes	Yes	9.00	早期學習階段
Task 3 1M	Yes	Yes	Yes	Yes	17.05	已學到基本策略
Task 3 1.5M	Yes	Yes	Yes	Yes	17.15	表現穩定但仍未通過
Task 3 2M	Yes	Yes	Yes	Yes	17.15	與 1.5M 接近
Task 3 2.5M	Yes	Yes	Yes	Yes	18.50	最佳 Task 3 snapshot，接近通過

從 limited ablation 結果可以看出，Task 3 的 enhanced DQN 隨著 training steps 增加而逐步改善。600k steps 時 mean reward 僅為 9.00，但到 1M steps 已提升到 17.05，最後 2.5M steps 達到 18.50。這表示 Double DQN、PER 與 n-step return 的組合在調參後具備學習能力，且較長訓練時間能持續改善策略。

然而，Task 3 最佳結果仍低於 Task 2 vanilla DQN 的 19.30。這說明 enhancement 本身不一定保證立即優於 baseline，尤其在 PER 與 multi-step return 同時使用時。本次結果顯示，Task 3 的主要限制可能不是方法無效，可能是超參數設定仍有進一步調整空間。

4. Additional Analysis

甲、Loss 的選定

在 Task 1 中，我使用 `MSELoss` 作為 DQN 的 `loss function`。由於 `CartPole` 的 `state space` 較小、`reward` 較穩定，因此 `MSELoss` 已足以訓練出有效 `policy`。

而在 Task 2 與 Task 3 的 `Pong` 環境中，我改用 `SmoothL1Loss`。`Pong` 的 `observation` 是高維度影像，且 `reward` 較不穩，因此 `TD error` 可能出現較大的波動。`SmoothL1Loss` 相較 `MSELoss` 對 `outlier` 較不敏感，可以降低過大 `TD error` 對 `gradient update` 的影響，使訓練更穩定。

乙、Gradient clipping

在 `Pong` 作業中，我加入 `gradient clipping`，將 `gradient norm` 限制在 10.0 以內。這樣做的原因是 `Atari` 環境的輸入維度較高，而且 `Q-value` 更新可能不穩定。若 `gradient` 過大，可能導致 `network` 參數更新幅度過大，進而使訓練發散。透過這方式，希望可以讓每次更新更穩定，降低訓練過程中的震盪。

丙、Evaluation frequency

我也根據環境複雜度調整 `evaluation frequency`。`CartPole` 的 `episode` 較短，`evaluation` 成本較低，因此訓練時 Task 1 每 20 `episodes` 進行一次 `evaluation`。`Pong` 的 `episode` 較長，且每一步都需要處理影像輸入，因此 `evaluation` 成本較高，所以 Task 2 與 Task 3 每 100 `episodes` 才進行一次 `evaluation`。這樣可以減少 `evaluation` 對整體訓練時間的影響。

5. Conclusion

本作業實作並比較了三個 `value-based reinforcement learning` 任務。Task 1 使用 `vanilla DQN` 解決 `CartPole-v1`，結果顯示模型能在低維度狀態空間中有效學習，最後 `evaluation reward` 達到 500，代表 `agent` 已能穩定完成整個 `episode`。

Task 2 將 DQN 應用到 `Atari Pong-v5`，並使 `grayscale`、`resize` 與 `CNN-based Q-network` 處理高維度影像輸入。從 `training curve` 可以觀察到，模型一開始 `reward` 接近 -20，但在約 1M `environment steps` 後快速提升，後期大多維持 17 到 19 左右，顯示 `vanilla DQN` 已能學到有效的策略。

Task 3 則進一步加入 Double DQN、Prioritized Experience Replay 與 multi-step return。初始設定下，Task 3 的訓練較不穩定，因此我進一步調整 learning rate、target update frequency、PER alpha/beta 與 n-step return。調參後，Task 3 的表現大幅改善，最佳 snapshot 出現在 2.5M environment steps，正式測試 mean reward 達到 18.50。雖然尚未達到 19 分門檻，但已非常接近 requirement，並且明顯優於早期 snapshot。

整體而言，vanilla DQN 在 CartPole 表現穩定，CNN-based DQN 也能在 Pong 中學到有效策略。然而，enhanced DQN 的效果仍需要透過更完整的 ablation study 與 hyperparameter tuning 進一步驗證。未來若能分別測試 Double DQN、PER 與 multi-step return 的個別貢獻，將能更清楚分析每項技術對 sample efficiency 的影響。